STM32L476RG and SSD1306 I2C OLED Display

Register-level tutorial with bare-metal C, I2C timing, OLED commands, text, logo, and animation

For classroom and lab use

Target Board: STM32L476RG / NUCLEO-L476RG
Display Hardware: SSD1306-style 0.96-inch 128 x 64 monochrome OLED module
Interface: I2C1
Pins Used: PB8 = SCL, PB9 = SDA, plus 3.3 V and GND

Figure 1: SSD1306-style OLED module with GND, VCC, SCL, and SDA pins

Hardware Used

Item	Quantity	Notes
STM32L476RG board	1	NUCLEO-L476RG or compatible board
SSD1306 OLED module	1	128 x 64, I2C version, usually address 0x3C
Jumper wires	4+	Short wires are easier to debug
USB cable	1	Programming and board power

Check before power: Confirm the OLED module pin order. Many modules use GND, VCC, SCL, SDA, but some boards swap the order.

Contents

1. Purpose of the tutorial

This tutorial shows how a 128 x 64 OLED display is driven from an STM32L476RG using I2C1 and direct register access. The focus is the path from register configuration to visible pixels: GPIO setup, I2C timing, SSD1306 command bytes, display data bytes, the screen buffer, and the display demos in main.c.

The project uses a layered structure. **main.c** selects what appears on the screen. **oled.c** creates the pixel data. **i2c.c** sends bytes over I2C. **gpio.c** configures PB8 and PB9 so the I2C peripheral can drive the bus.

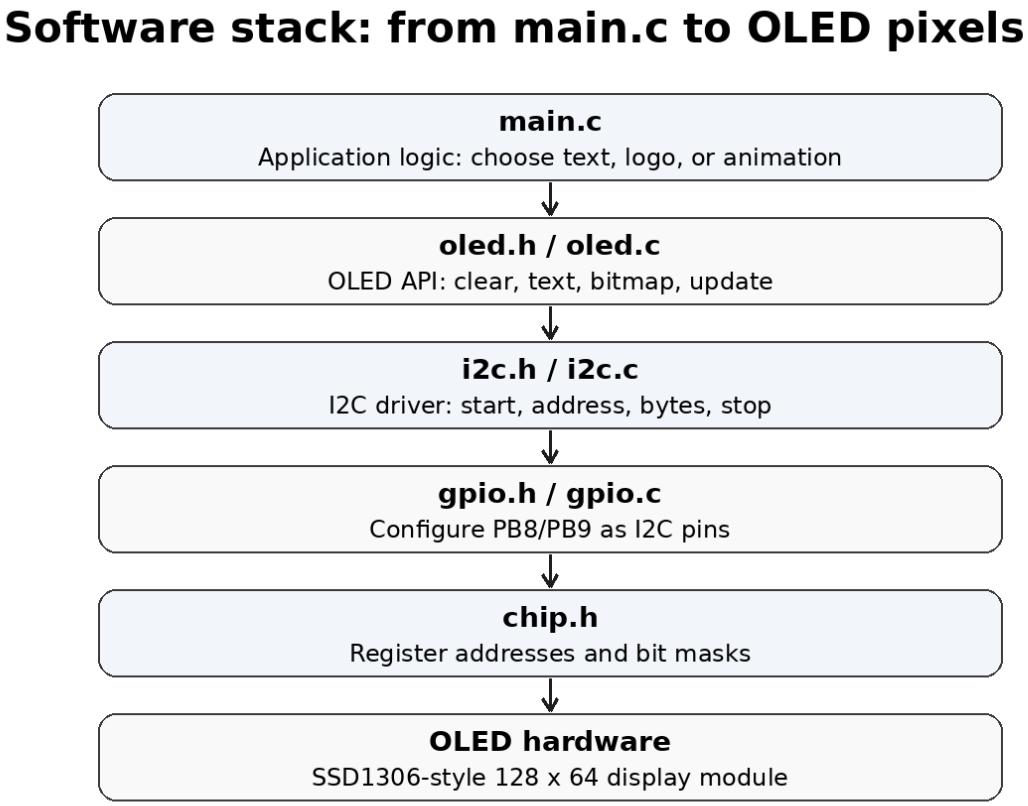

Figure 2: Software stack from main.c to the OLED hardware

2. Display overview

The display is a small monochrome OLED module using an SSD1306-style controller. The common module size is 0.96 inch with a 128 x 64 pixel matrix. OLED pixels emit their own light, so there is no backlight to control. Black pixels are off, and white pixels are on.

Feature	Value used here	Why it matters
Resolution	128 x 64 pixels	The frame buffer is 128 x 64 / 8 = 1024 bytes.
Color depth	1 bit per pixel	Each pixel is either black or white.
Interface	I2C	Only SCL and SDA are needed for display communication.
OLED address	0x3C	Defined in oled.h as OLED_ADDR. Some modules can use 0x3D.
Control byte for commands	0x00	The next byte is interpreted as an SSD1306 command.
Control byte for display data	0x40	The following bytes are written into display RAM.

The SSD1306 data sheet describes the I2C slave address format as 0111100 or 0111101 depending on SA0. With SA0 low, the 7-bit address is 0x3C. The same data sheet also defines the control byte: D/C# = 0 selects a command, and D/C# = 1 selects display data.

3. Wiring

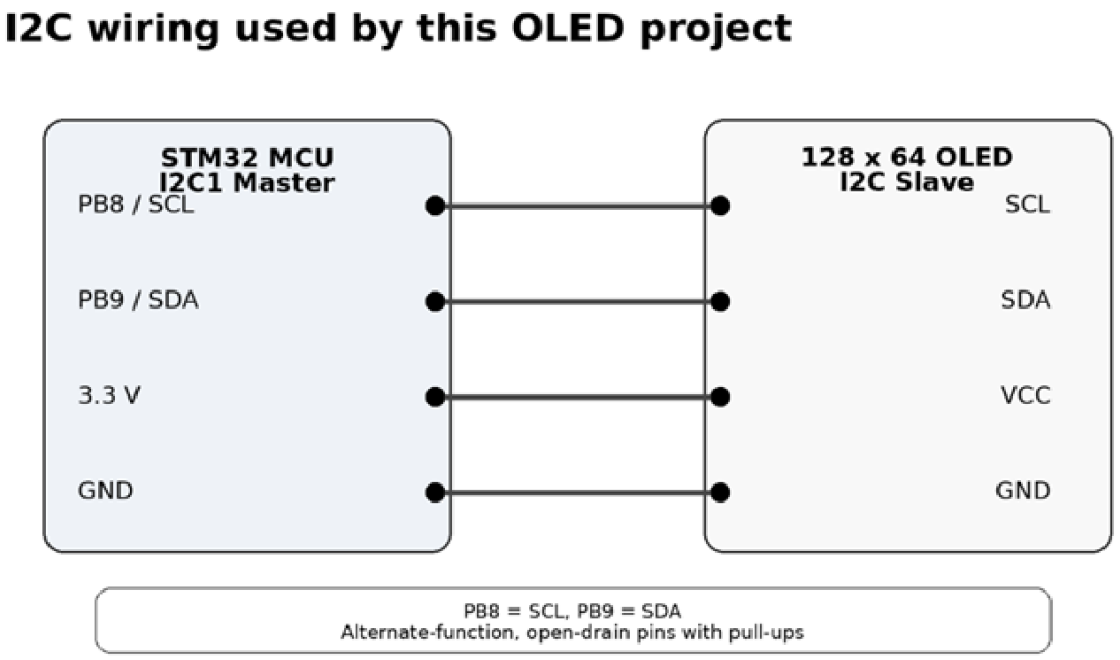

Figure 3: I2C wiring used by this OLED project

OLED pin	STM32 connection	Code setting
GND	Board GND	Common reference for the bus
VCC	3.3 V	Most modules also tolerate 5 V when a regulator is built in; use 3.3 V for this setup.
SCL	PB8 / I2C1_SCL	Alternate function AF4, open-drain, pull-up
SDA	PB9 / I2C1_SDA	Alternate function AF4, open-drain, pull-up

Important wiring rule: I2C lines are not push-pull lines. SCL and SDA are released high through pull-ups and pulled low by devices on the bus.

4. Program structure

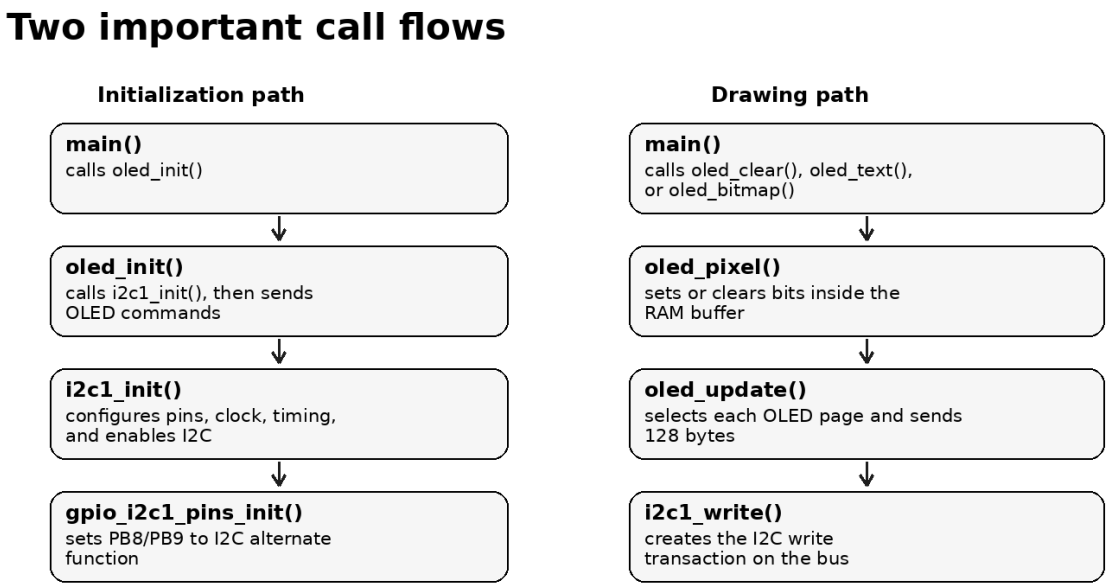

Figure 4: Initialization path and drawing path

File	Role	Main idea
chip.h	Register map	Defines register structures, peripheral base addresses, and bit masks.
gpio.c / gpio.h	Pin setup	Configures PB8 and PB9 for I2C1.
i2c.c / i2c.h	I2C driver	Initializes I2C1 and sends write transactions.
oled.h	Display API	Defines OLED_ADDR, OLED_WIDTH, OLED_HEIGHT, colors, and drawing functions.
oled.c	OLED driver	Sends SSD1306 commands, stores the RAM buffer, draws text, pixels, and bitmaps.
oled_fonts.c / .h	Fonts	Stores bitmap fonts used by oled_text().
oled_bitmap.h	Logo bitmap	Stores the 128 x 64 logo image array.
oled_horse.h	Animation frames	Stores bitmap frames horse1 through horse10.
main.c	Demo code	Runs the display examples. Only one endless display loop should be active at a time.

5. Register map used in the code

chip.h maps STM32 peripheral base addresses to C structures. This makes register access readable. For example, GPIOB->MODER writes to address 0x48000400, and I2C1->TIMINGR writes to address 0x40005410 because TIMINGR is the fifth 32-bit register in the i2c_regs_t structure.

Peripheral/register	Address or offset	Used for
RCC base	0x40021000	Clock control and peripheral reset registers
GPIOB base	0x48000400	PB8 and PB9 mode, output type, speed, pull-up, alternate function
I2C1 base	0x40005400	I2C control, timing, status, clear flags, transmit data
RCC_AHB2ENR	RCC + 0x4C	GPIOB clock enable, bit 1
RCC_APB1ENR1	RCC + 0x58	I2C1 clock enable, bit 21
RCC_CCIPR	RCC + 0x88	I2C1 clock source selection, bits 13:12
GPIOB_MODER	GPIOB + 0x00	PB8/PB9 mode: 10 = alternate function
GPIOB_OTYPER	GPIOB + 0x04	PB8/PB9 output type: 1 = open-drain
GPIOB_OSPEEDR	GPIOB + 0x08	PB8/PB9 output speed: 10 = fast speed
GPIOB_PUPDR	GPIOB + 0x0C	PB8/PB9 pull-up: 01 = pull-up
GPIOB_AFR[1]	GPIOB + 0x24	PB8/PB9 alternate function number, AF4 = I2C1

I2C1_CR1	I2C1 + 0x00	Peripheral enable
I2C1_CR2	I2C1 + 0x04	Address, transfer length, START, AUTOEND
I2C1_TIMINGR	I2C1 + 0x10	SCL high and low timing
I2C1_ISR	I2C1 + 0x18	BUSY, TXIS, NACKF, STOPF status flags
I2C1_ICR	I2C1 + 0x1C	Clear STOP, NACK, bus error, arbitration lost, overrun flags
I2C1_TXDR	I2C1 + 0x28	Transmit data byte

Clock first: STM32 peripherals do not respond correctly until their peripheral clock is enabled. That is why GPIOB and I2C1 clocks are enabled before configuring their registers.

6. GPIO setup for PB8 and PB9

PB8 and PB9 are normal GPIO pins after reset. The code changes them into I2C1 pins by selecting alternate function mode and AF4. The same function also sets open-drain operation and pull-ups, which match the electrical behavior of the I2C bus.

gpio.c - gpio_i2c1_pins_init()

```
#include "chip.h"
#include "gpio.h"

void gpio_i2c1_pins_init(void)
{
    RCC->AHB2ENR |= RCC_AHB2ENR_GPIOBEN;

    /* PB8 = I2C1_SCL, PB9 = I2C1_SDA */
    GPIOB->MODER &= ~((3UL << (8U * 2U)) | (3UL << (9U * 2U)));
    GPIOB->MODER |=  ((2UL << (8U * 2U)) | (2UL << (9U * 2U)));

    GPIOB->OTYPER |=  (1UL << 8U) | (1UL << 9U);

    GPIOB->OSPEEDR &= ~((3UL << (8U * 2U)) | (3UL << (9U * 2U)));
    GPIOB->OSPEEDR |=  ((2UL << (8U * 2U)) | (2UL << (9U * 2U)));

    GPIOB->PUPDR &= ~((3UL << (8U * 2U)) | (3UL << (9U * 2U)));
    GPIOB->PUPDR |=  ((1UL << (8U * 2U)) | (1UL << (9U * 2U)));

    GPIOB->AFR[1] &= ~((0xFUL << 0U) | (0xFUL << 4U));
    GPIOB->AFR[1] |=  ((4UL   << 0U) | (4UL   << 4U));
}
```

Code line	Register action	Meaning
RCC->AHB2ENR |= RCC_AHB2ENR_GPIOBEN;	Set RCC_AHB2ENR bit 1	Turns on GPIOB peripheral clock.
GPIOB->MODER ... 2UL	PB8/PB9 MODER = 10	Selects alternate function mode.
GPIOB->OTYPER |= ...	PB8/PB9 OTYPER = 1	Selects open-drain output.
GPIOB->OSPEEDR ... 2UL	PB8/PB9 OSPEEDR = 10	Fast output edges help meet I2C rise/fall timing.
GPIOB->PUPDR ... 1UL	PB8/PB9 PUPDR = 01	Enables internal pull-ups. External pull-ups on the OLED module may also be present.
GPIOB->AFR[1] ... 4UL	PB8/PB9 AFR = AF4	Connects the pins to the I2C1 peripheral.

Why open-drain? I2C devices pull the bus low but do not drive it high. A high level appears when all devices release the line and the pull-up resistor brings it back to VCC. This allows more than one device to share the same bus safely.

7. I2C1 setup

The I2C initialization function prepares the pins, turns on the I2C1 clock, selects the clock source, resets the peripheral, programs the timing register, clears old flags, and enables I2C1.

i2c.c - i2c1_init()

```
void i2c1_init(void)
{
    gpio_i2c1_pins_init();
```

```
    RCC->APB1ENR1 |= RCC_APB1ENR1_I2C1EN;

    /* I2C1 clock source = SYSCLK, keep default MSI = 4 MHz */
    RCC->CCIPR &= ~(3UL << 12U);
    RCC->CCIPR |=  (1UL << 12U);

    RCC->APB1RSTR1 |=  RCC_APB1RSTR1_I2C1RST;
    RCC->APB1RSTR1 &= ~RCC_APB1RSTR1_I2C1RST;

    I2C1->CR1 &= ~I2C_CR1_PE;

    /* 100 kHz, source clock = 4 MHz */
    I2C1->TIMINGR = 0x00000E14UL;

    I2C1->ICR = I2C_ICR_STOPCF | I2C_ICR_NACKCF | I2C_ICR_BERRCF |
                I2C_ICR_ARLOCF | I2C_ICR_OVRCF;

    I2C1->CR1 = I2C_CR1_PE;
}
```

1. Configure PB8 and PB9 by calling gpio_i2c1_pins_init().
2. Enable the I2C1 peripheral clock in RCC_APB1ENR1.
3. Select SYSCLK as the I2C1 clock source using RCC_CCIPR bits 13:12.
4. Reset I2C1 through RCC_APB1RSTR1 so the peripheral starts from a known state.
5. Disable I2C1 before changing the timing register. TIMINGR is configured while PE = 0.
6. Write TIMINGR = 0x00000E14 for a Standard-mode bus target near 100 kHz with the default 4 MHz MSI clock.
7. Clear STOP, NACK, bus error, arbitration lost, and overrun flags.
8. Enable the peripheral by setting I2C_CR1_PE.

8. I2C speed, timing, and signal rules

Why this tutorial uses 100 kHz

The code uses Standard-mode I2C. Standard-mode is a conservative choice for a small OLED module on jumper wires because it gives slower edges more time to settle. It also keeps the setup simple when the pull-up value on the OLED module is unknown. Fast-mode at 400 kHz can work, but it depends more heavily on short wiring, correct pull-ups, and clean rise time.

Mode	Typical maximum bit rate	Use here
Standard-mode	100 kbit/s	Used by this code. Stable and easy to debug.
Fast-mode	400 kbit/s	Possible when wiring and pull-ups are good.
Fast-mode Plus	1 Mbit/s	Requires stronger electrical setup and Fm+ capable pins/settings.

Full-screen update estimate. A 128 x 64 monochrome buffer contains 1024 data bytes. At 100 kbit/s, one full screen transfer is roughly 1024 bytes x 9 clock bits per byte = 9216 clocks, plus address, control bytes, commands, and STOP/START overhead. That is fast enough for text, logos, and simple bitmap animation.

TIMINGR = 0x00000E14

The I2C timing register divides the I2C kernel clock into SCL low time, SCL high time, and data setup/hold timing. The comment in i2c.c says the I2C clock source is SYSCLK while the default MSI clock remains 4 MHz. With that clock, each timing tick is about 250 ns when PRESC = 0.

TIMINGR field	Bits	Value from 0x00000E14	Meaning in this setup
PRESC	31:28	0x0	No extra prescaler. Timing tick is about 250 ns with 4 MHz I2C clock.
SCLDEL	23:20	0x0	Data setup delay field left at zero for this simple Standard-mode setup.
SDADEL	19:16	0x0	Data hold delay field left at zero.

SCLH	15:8	0x0E	SCL high counter. About 15 ticks before analog/filter effects.
SCLL	7:0	0x14	SCL low counter. About 21 ticks before analog/filter effects.

The raw counter estimate is close to Standard-mode: SCL low is about 5.25 us and SCL high is about 3.75 us before adding real bus rise time and filter delays. The practical target is near 100 kHz, which matches the comment in the code and the Standard-mode range in the I2C specification.

Do not change TIMINGR blindly: TIMINGR depends on the I2C kernel clock, rise time, fall time, filter settings, and target speed. If SYSCLK changes from the default 4 MHz MSI clock, recalculate the timing value.

Polarity and data validity

Signal rule	Meaning
Idle state	SCL = HIGH and SDA = HIGH. The bus is free.
START	SDA goes HIGH to LOW while SCL is HIGH.
STOP	SDA goes LOW to HIGH while SCL is HIGH.
Data bit	SDA is sampled while SCL is HIGH. SDA should change only while SCL is LOW.
ACK	The receiver pulls SDA LOW during the ninth clock.
No SPI-style CPOL/CPHA	I2C does not use SPI clock-polarity and phase settings. The idle-high bus and START/STOP rules define the signaling.

9. I2C write transaction

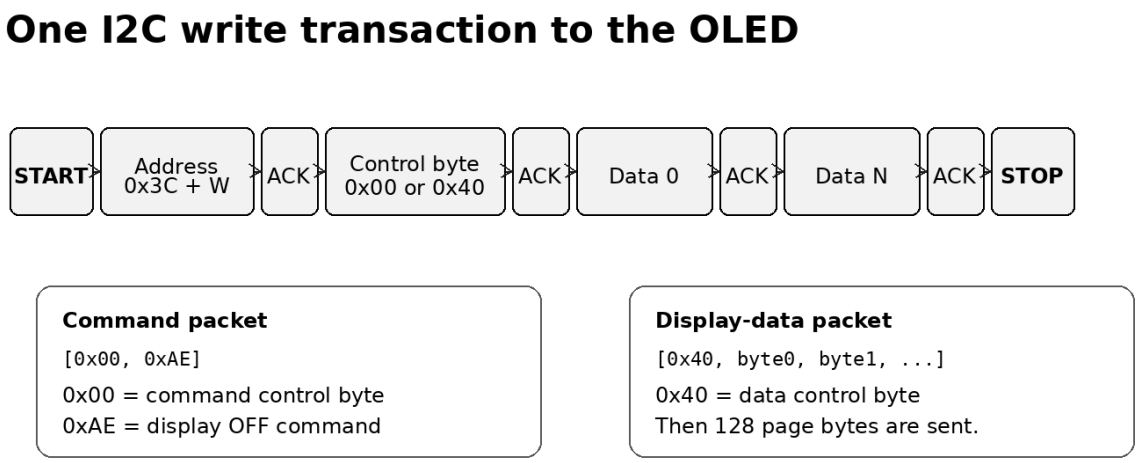

Figure 5: One I2C write transaction to the OLED

The write function waits for an idle bus, clears old flags, loads the 7-bit address, loads the number of bytes, enables AUTOEND, generates START, writes each byte into TXDR, waits for STOPF, then clears the STOP flag.

i2c.c - i2c1_write()

```
int i2c1_write(uint8_t addr7, const uint8_t *data, uint16_t len)
{
    uint16_t i;

    if ((data == 0) || (len == 0U) || (len > 255U)) {
        return -1;
    }

    if (wait_while_set(&I2C1->ISR, I2C_ISR_BUSY) < 0) {
        return -1;
    }
```

```
    I2C1->ICR = I2C_ICR_STOPCF | I2C_ICR_NACKCF | I2C_ICR_BERRCF |
                I2C_ICR_ARLOCF | I2C_ICR_OVRCF;

    I2C1->CR2 = ((uint32_t)addr7 << 1U) |
                ((uint32_t)len   << 16U) |
                I2C_CR2_AUTOEND;

    I2C1->CR2 |= I2C_CR2_START;

    for (i = 0U; i < len; i++) {
        if (wait_until_set(&I2C1->ISR, I2C_ISR_TXIS | I2C_ISR_NACKF) < 0) {
            return -1;
        }

        if ((I2C1->ISR & I2C_ISR_NACKF) != 0U) {
            I2C1->ICR = I2C_ICR_NACKCF | I2C_ICR_STOPCF;
            return -1;
        }

        I2C1->TXDR = data[i];
    }

    if (wait_until_set(&I2C1->ISR, I2C_ISR_STOPF) < 0) {
        return -1;
    }

    I2C1->ICR = I2C_ICR_STOPCF;
    return 0;
}
```

Step	Register/flag	What happens
1	I2C_ISR_BUSY	Wait until the bus is free.
2	I2C_ICR flags	Clear old STOP, NACK, bus error, arbitration lost, and overrun flags.
3	I2C_CR2.SADD	Load the 7-bit OLED address shifted left by one bit.
4	I2C_CR2.NBYTES	Load the number of bytes to transmit. The code limits this to 255 bytes.
5	I2C_CR2.AUTOEND	Tell the peripheral to generate STOP automatically after NBYTES.
6	I2C_CR2.START	Generate START and address the OLED for writing.
7	I2C_ISR_TXIS	Wait until TXDR can accept the next byte.
8	I2C1->TXDR	Write one byte to the bus.
9	I2C_ISR_NACKF	Abort if the OLED does not acknowledge.
10	I2C_ISR_STOPF	Wait for the automatic STOP, then clear STOPCF.

10. SSD1306 command/data protocol

The OLED receives two kinds of information over the same I2C bus. A control byte tells the SSD1306 how to interpret the following byte or bytes.

oled.c - control bytes

```
#define OLED_CMD  0x00
#define OLED_DATA 0x40
#define OLED_PAGES (OLED_HEIGHT / 8)
```

Packet type	First byte	Following bytes	Example
Command packet	0x00	SSD1306 command byte	[0x00, 0xAE] turns the display off
Data packet	0x40	Display RAM bytes	[0x40, byte0, byte1, ...] writes pixels

oled_command(value) sends one command by calling i2c1_write_byte(OLED_ADDR, OLED_CMD, value).
oled_data_block(data, len) builds a packet where packet[0] is 0x40 and the rest of the packet is display RAM data.

oled.c - command and data helpers

```
static int oled_command(uint8_t value)
{
    return i2c1_write_byte(OLED_ADDR, OLED_CMD, value);
}
```

```
static int oled_data_block(const uint8_t *data, uint16_t len)
{
    uint8_t packet[1 + OLED_WIDTH];
    if (len > OLED_WIDTH) {
        return -1;
    }

    packet[0] = OLED_DATA;
    memcpy(&packet[1], data, len);
    return i2c1_write(OLED_ADDR, packet, (uint16_t)(len + 1U));
}
```

Initialization command sequence

oled_init() sends a sequence of SSD1306 commands before normal drawing begins. The sequence turns the display off, selects memory mode, sets remap and scan direction, configures multiplexing, contrast, charge pump, and finally turns the display on.

Command	Name / role	Why it is used
0xAE	Display OFF	Keeps the panel off while configuration changes are sent.
0x20, 0x10	Memory addressing mode	Selects the addressing behavior used by the driver.
0xB0	Page start address	Starts from page 0.
0xC8	COM scan direction	Matches the physical module orientation.
0x00, 0x10	Column address nibbles	Sets the starting column address.
0x40	Display start line	Starts at line 0.
0x81, 0xFF	Contrast control	Sets high contrast for readability.
0xA1	Segment remap	Matches the horizontal direction of the module.
0xA6	Normal display	White pixels are on, black pixels are off.
0xA8, 0x3F	Multiplex ratio	Sets 64 multiplex lines for a 128 x 64 panel.
0xD3, 0x00	Display offset	No vertical offset.
0xD5, 0xF0	Display clock divide / oscillator	Configures SSD1306 display clock.
0xD9, 0x22	Pre-charge period	OLED panel drive setting.
0xDA, 0x12	COM pins hardware config	Common value for 128 x 64 modules.
0xDB, 0x20	VCOMH deselect level	OLED panel voltage setting.
0x8D, 0x14	Charge pump	Enables internal charge pump.
0xAF	Display ON	Starts displaying RAM content.

11. OLED buffer, pages, pixels, text, and bitmaps

The OLED is not updated pixel by pixel directly. The driver first edits a RAM buffer inside the STM32. After the buffer is ready, oled_update() sends the buffer to the display over I2C.

Concept	Value in this project	Explanation
OLED width	128	Number of columns.
OLED height	64	Number of rows.
Page height	8 pixels	SSD1306 page addressing stores 8 vertical pixels per byte.
Number of pages	64 / 8 = 8	Pages 0 through 7.
Buffer size	128 x 8 = 1024 bytes	One byte per column per page.

oled.c - display buffer

```
static uint8_t oled_buffer[OLED_WIDTH * OLED_PAGES];
static uint16_t cursor_x;
static uint16_t cursor_y;
```

Pixel mapping. For a pixel at coordinate (x, y), page = y / 8 and bit = y % 8. Setting that bit turns the pixel white. Clearing that bit turns it black.

oled.c - oled_pixel()

```
void oled_pixel(uint16_t x, uint16_t y, oled_color_t color)
{
    if ((x >= OLED_WIDTH) || (y >= OLED_HEIGHT)) {
        return;
    }
```

```
    if (color == OLED_WHITE) {
        oled_buffer[x + (y / 8U) * OLED_WIDTH] |=  (1U << (y % 8U));
    } else {
        oled_buffer[x + (y / 8U) * OLED_WIDTH] &= ~(1U << (y % 8U));
    }
}
```

oled_update() sends eight pages. For each page, it selects the page address, resets the column address, and sends 128 bytes from oled_buffer. One screen refresh therefore sends 8 x 128 = 1024 display bytes, plus command and control bytes.

oled.c - oled_update()

```
void oled_update(void)
{
    uint8_t page;

    for (page = 0U; page < OLED_PAGES; page++) {
        oled_command((uint8_t)(0xB0U + page));
        oled_command(0x00);
        oled_command(0x10);
        oled_data_block(&oled_buffer[OLED_WIDTH * page], OLED_WIDTH);
    }
}
```

Text and bitmaps use the same rule: draw into the RAM buffer first, then call oled_update(). **oled_text()** writes character pixels using the font tables. **oled_bitmap()** copies bitmap pixels into the buffer using the bitmap arrays from oled_bitmap.h and oled_horse.h.

12. main.c display demos

main.c contains separate display examples. Use one endless display loop at a time. The logo loop is active in the uploaded file. The Hello and horse blocks are inside comments.

> **Main rule:** Leave only one endless loop active. If two while (1) blocks are active, the first one prevents the code below it from running.

Current active block: SSU logo

This block clears the display white, draws the logo in black, waits, then clears the display black and draws the logo in white. That creates an inverted blinking effect.

main.c - SSU logo block

```
    //SSU Logo
while (1){
    oled_clear(OLED_WHITE);

    oled_bitmap(0, 0, logo, 128, 64, OLED_BLACK);
    oled_update();
    for (int i=0;i<200;i++){
            delay();
    }
    oled_clear(OLED_BLACK);

    oled_bitmap(0, 0, logo, 128, 64, OLED_WHITE);
    oled_update();
}
```

Hello text block

To display Hello, comment out the SSU logo loop, then remove the comment markers around the Hello block. The flow is simple: clear the buffer, write text, update the display, then stop inside while (1).

main.c - Hello block

```
//Uncomment this section if you want to Print Hello
/*
```

```
    oled_clear(OLED_BLACK);
    oled_text("Hello", &font_16x26, OLED_WHITE);
    oled_update();
    while (1);
*/
```

Horse animation block

To run the horse animation, comment out the SSU logo loop and uncomment the horse block. Each frame is a full 128 x 64 bitmap. The code clears the buffer, draws one frame, updates the display, delays, then moves to the next frame.

main.c - Horse animation block

```
        //This Part is for displaying a Horse running
        /*

    while (1)
    {
        oled_clear(OLED_BLACK);
        oled_bitmap(0, 0, horse2, 128, 64, OLED_WHITE);
        oled_update();

        delay();

        oled_clear(OLED_BLACK);
        oled_bitmap(0, 0, horse3, 128, 64, OLED_WHITE);
        oled_update();

        delay();
        oled_clear(OLED_BLACK);
        oled_bitmap(0, 0, horse4, 128, 64, OLED_WHITE);
        oled_update();

        delay();

        oled_clear(OLED_BLACK);
        oled_bitmap(0, 0, horse5, 128, 64, OLED_WHITE);
        oled_update();

        delay();

        oled_clear(OLED_BLACK);
        oled_bitmap(0, 0, horse6, 128, 64, OLED_WHITE);
        oled_update();

        delay();

        oled_clear(OLED_BLACK);
        oled_bitmap(0, 0, horse7, 128, 64, OLED_WHITE);
        oled_update();

        delay();
        oled_clear(OLED_BLACK);
        oled_bitmap(0, 0, horse8, 128, 64, OLED_WHITE);
        oled_update();

        delay();
        oled_clear(OLED_BLACK);
        oled_bitmap(0, 0, horse9, 128, 64, OLED_WHITE);
        oled_update();

        delay();
        oled_clear(OLED_BLACK);
        oled_bitmap(0, 0, horse10, 128, 64, OLED_WHITE);
        oled_update();

        delay();
    }*/
```

Animation timing. The delay() function is a short software loop. More delay makes the animation slower. Less delay makes it faster. Since this is a blocking delay, the CPU waits during the loop.

13. Troubleshooting

Problem	Most likely cause	Fix

Screen is blank	Wrong wiring, no power, wrong address, or I2C not enabled	Check GND/VCC/SCL/SDA, confirm address 0x3C, verify oled_init() is called.
Only random pixels appear	Buffer data is being sent but command setup or addressing is wrong	Check the SSD1306 initialization sequence and page update commands.
Image is mirrored or upside down	Segment remap or COM scan direction does not match the module	Check commands 0xA1 and 0xC8.
I2C write returns error	NACK or bus stuck busy	Check address, pull-ups, wiring, and whether another device is holding SDA/SCL low.
Text appears but bitmap does not	Bitmap dimensions or array format mismatch	Confirm the bitmap is 128 x 64 and passed with width 128, height 64.
Animation is too slow	Full-screen transfer plus software delay	Reduce delay or use a faster I2C timing only after checking bus rise time.
Nothing below a demo runs	An active while (1) loop never exits	Keep only the selected display loop active.

14. Appendix: source code excerpts

The following excerpts are taken from the provided project files and are included here so the register writes can be read next to the explanation.

oled.h

```
#ifndef OLED_H
#define OLED_H

#include <stdint.h>
#include "oled_fonts.h"

#define OLED_ADDR   0x3C
#define OLED_WIDTH  128
#define OLED_HEIGHT 64

typedef enum {
    OLED_BLACK = 0,
    OLED_WHITE = 1
} oled_color_t;

int  oled_init(void);
void oled_update(void);
void oled_clear(oled_color_t color);
void oled_pixel(uint16_t x, uint16_t y, oled_color_t color);
void oled_cursor(uint16_t x, uint16_t y);
char oled_char(char c, const oled_font_t *font, oled_color_t color);
char oled_text(const char *text, const oled_font_t *font, oled_color_t color);
void oled_bitmap(uint16_t x, uint16_t y, const unsigned char *bitmap, uint16_t w, uint16_t h, oled_color_t color);

#endif
```

i2c1_init()

```
void i2c1_init(void)
{
    gpio_i2c1_pins_init();

    RCC->APB1ENR1 |= RCC_APB1ENR1_I2C1EN;

    /* I2C1 clock source = SYSCLK, keep default MSI = 4 MHz */
    RCC->CCIPR &= ~(3UL << 12U);
    RCC->CCIPR |=  (1UL << 12U);

    RCC->APB1RSTR1 |=  RCC_APB1RSTR1_I2C1RST;
    RCC->APB1RSTR1 &= ~RCC_APB1RSTR1_I2C1RST;

    I2C1->CR1 &= ~I2C_CR1_PE;

    /* 100 kHz, source clock = 4 MHz */
    I2C1->TIMINGR = 0x00000E14UL;

    I2C1->ICR = I2C_ICR_STOPCF | I2C_ICR_NACKCF | I2C_ICR_BERRCF |
                I2C_ICR_ARLOCF | I2C_ICR_OVRCF;
```

```
    I2C1->CR1 = I2C_CR1_PE;
}
```

oled_init() command sequence

```
int oled_init(void)
{
    i2c1_init();

    if (oled_command(0xAE) < 0) return -1;
    if (oled_command(0x20) < 0) return -1;
    if (oled_command(0x10) < 0) return -1;
    if (oled_command(0xB0) < 0) return -1;
    if (oled_command(0xC8) < 0) return -1;
    if (oled_command(0x00) < 0) return -1;
    if (oled_command(0x10) < 0) return -1;
    if (oled_command(0x40) < 0) return -1;
    if (oled_command(0x81) < 0) return -1;
    if (oled_command(0xFF) < 0) return -1;
    if (oled_command(0xA1) < 0) return -1;
    if (oled_command(0xA6) < 0) return -1;
    if (oled_command(0xA8) < 0) return -1;
    if (oled_command(0x3F) < 0) return -1;
    if (oled_command(0xA4) < 0) return -1;
    if (oled_command(0xD3) < 0) return -1;
    if (oled_command(0x00) < 0) return -1;
    if (oled_command(0xD5) < 0) return -1;
    if (oled_command(0xF0) < 0) return -1;
    if (oled_command(0xD9) < 0) return -1;
    if (oled_command(0x22) < 0) return -1;
    if (oled_command(0xDA) < 0) return -1;
    if (oled_command(0x12) < 0) return -1;
    if (oled_command(0xDB) < 0) return -1;
    if (oled_command(0x20) < 0) return -1;
    if (oled_command(0x8D) < 0) return -1;
    if (oled_command(0x14) < 0) return -1;
    if (oled_command(0xAF) < 0) return -1;

    oled_clear(OLED_BLACK);
    oled_update();
    cursor_x = 0U;
    cursor_y = 0U;
    return 0;
}
```

15. References

[1] STMicroelectronics. RM0351 Reference Manual: STM32L4x5 and STM32L4x6 advanced Arm-based 32-bit MCUs. Register descriptions for RCC, GPIO, and I2C. https://www.st.com/resource/en/reference_manual/dm00083560.pdf

[2] STMicroelectronics. DS10198 Datasheet: STM32L476xx ultra-low-power Arm Cortex-M4 MCU. Electrical characteristics, supply range, alternate functions, and I2C capabilities. https://www.st.com/resource/en/datasheet/stm32l476je.pdf

[3] Solomon Systech. SSD1306 OLED/PLED Segment/Common Driver with Controller Data Sheet. I2C address format, control bytes, commands, and GDDRAM organization. https://cdn-shop.adafruit.com/datasheets/SSD1306.pdf

[4] NXP Semiconductors. UM10204 I2C-bus specification and user manual. Standard-mode, Fast-mode, open-drain bus behavior, START/STOP, ACK, and data-valid timing. https://www.nxp.com/documents/user_manual/UM10204.pdf

[5] Controllerstech. How to Interface SSD1306 OLED Display with STM32 Using I2C. Display overview, wiring concept, and SSD1306 STM32 workflow. https://controllerstech.com/oled-display-using-i2c-stm32/